Bellman-Ford-Moore

Cinco implementaciones, de la matriz a la cola

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre de 2026

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

Objetivos de la sesión

Al terminar la clase, usted podrá

- Calcular a mano los caminos más cortos de a lo sumo k aristas y relacionarlos con las rondas de Bellman-Ford.
- Leer cinco implementaciones en C++ de Bellman-Ford-Moore y decir qué cambia de una a la siguiente.
- Dar el costo en tiempo y en espacio de cada versión.

Y también

- Recuperar un ciclo negativo a partir de p.
- Seguir en una tabla la versión con cola y explicar `enCola` y `conteo`.
- Modelar un problema como un grafo de estados y resolverlo con la búsqueda en amplitud.

La pregunta de hoy

Lo que ya se tiene

Bellman-Ford hace $n - 1$ rondas; en cada una relaja todas las aristas. Una ronda más dice si hay un ciclo negativo alcanzable desde la fuente.

La pregunta

Esa idea cabe en un programa de cuarenta líneas, pero hay muchas formas de escribirlo. ¿Qué hay que guardar de una ronda a la siguiente? ¿Hace falta hacer siempre las $n - 1$ rondas? ¿Hace falta relajar todas las aristas en cada una?

El plan

Cinco versiones, de la que más gasta a la que menos: cada una corrige algo de la anterior.

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

La idea, en una recurrencia

Definición

Sea $G = (V, E)$ dirigido con pesos w y fuente s, con $n = |V|$. Para $k \geq 0$, $d_k(v)$ es el menor peso de un camino de s a v con a lo sumo k aristas, o ∞ si no hay ninguno.

Cómo se calcula la fila k con la fila $k-1$

$$d_0(s) = 0, \qquad d_0(v) = \infty \text{ para } v \neq s,$$

$$d_k(v) = \text{mín}\left(d_{k-1}(v), \min_{(u,v)\in E}\left(d_{k-1}(u) + w(u,v)\right)\right).$$

Un camino de a lo sumo k aristas que llega a v tiene a lo sumo $k-1$, o termina en una arista (u, v) después de un camino de a lo sumo $k-1$ aristas hasta u.

La idea, en una recurrencia

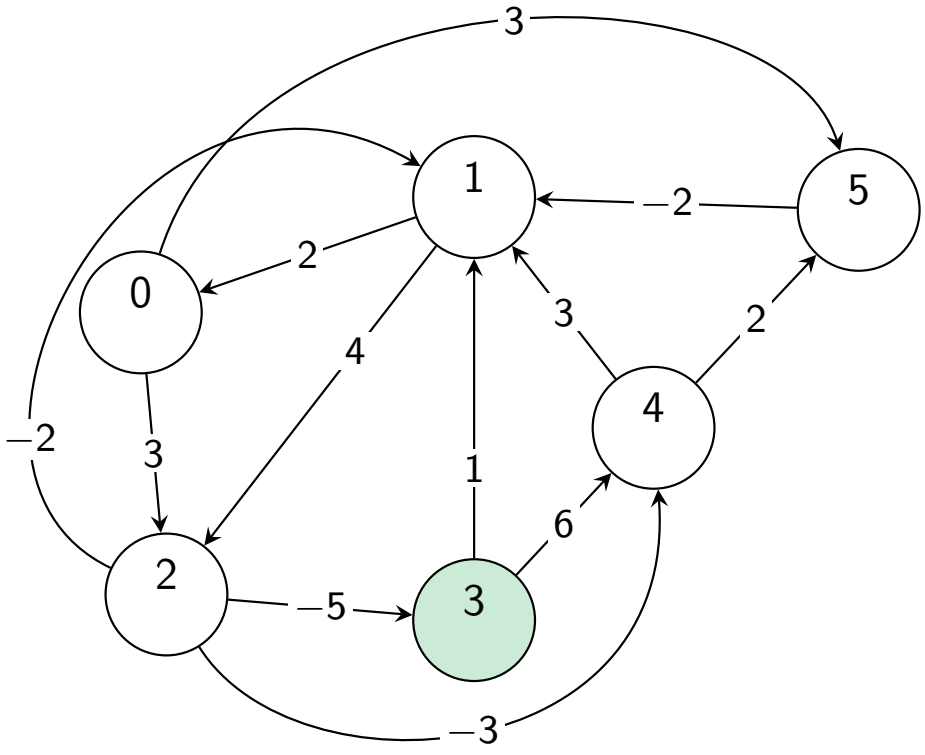
Por qué basta llegar a $k = n - 1$

Si no hay ciclos negativos alcanzables desde s, para cada v hay un camino más corto simple, y uno simple tiene a lo sumo $n - 1$ aristas. Entonces $d_{n-1}(v) = \delta(s, v)$ (CLRS, Sección 24.1, Lema 24.2).

En el grafo del ejemplo

El ejemplo que sigue tiene 6 vértices: con las filas 0 a 5 ya están todas las distancias.

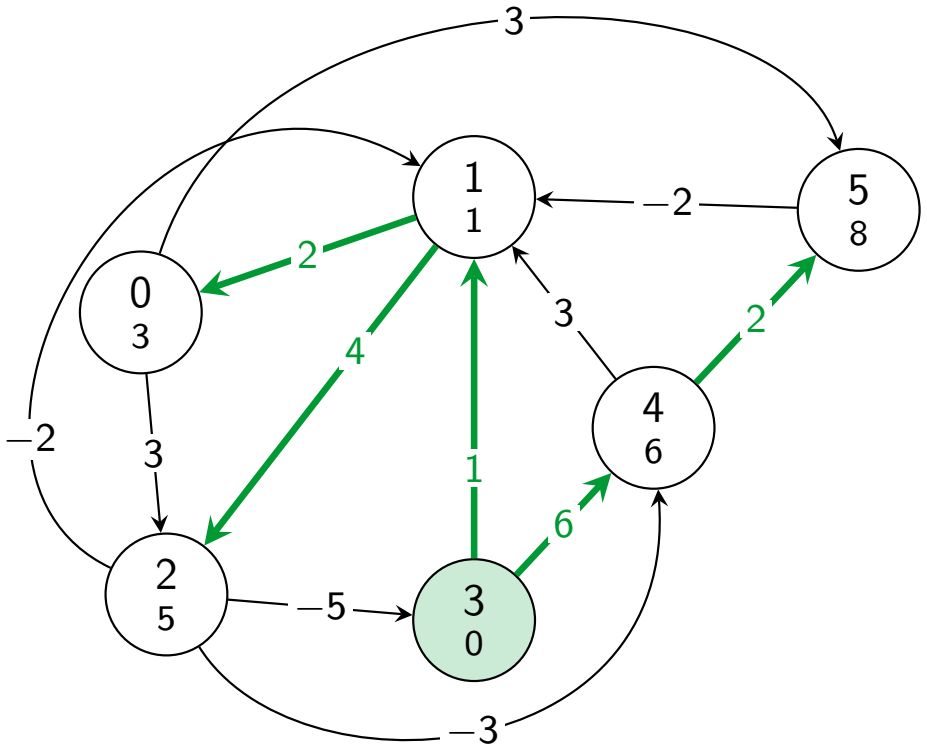
El grafo del ejemplo



Pregunta

La fuente es 3. ¿Cuánto cuesta llegar a 5 con a lo sumo dos aristas? ¿Y con a lo sumo cuatro?

El grafo del ejemplo



Respuesta

Con dos aristas el único camino es $\langle 3, 4, 5 \rangle$, de peso 8.

Con cuatro

$\langle 3, 1, 2, 4, 5 \rangle$ pesa $1 + 4 - 3 + 2 = 4$: la arista negativa $(2, 4)$ compensa las dos aristas de más.

Las filas, de 0 a 5 aristas

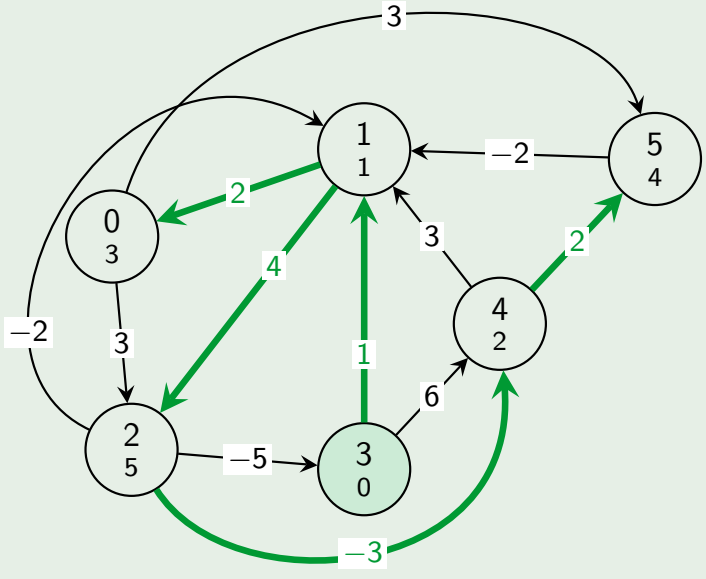
k	d_k						p					
	0	1	2	3	4	5	0	1	2	3	4	5
0	∞	∞	∞	0	∞	∞	-1	-1	-1	-1	-1	-1
1	∞	1	∞	0	6	∞	-1	3	-1	-1	3	-1
2	3	1	5	0	6	8	1	3	1	-1	3	4
3	3	1	5	0	2	6	1	3	1	-1	2	0
4	3	1	5	0	2	4	1	3	1	-1	2	4
5	3	1	5	0	2	4	1	3	1	-1	2	4

Cómo se lee

En la fila 3, 4 baja de 6 a 2 por $\langle 3, 1, 2, 4\rangle$ y 5 llega a 6 por $\langle 3, 1, 0, 5\rangle$. En la fila 4, 5 baja a 4 por $\langle 3, 1, 2, 4, 5\rangle$. La fila 5 repite la 4.

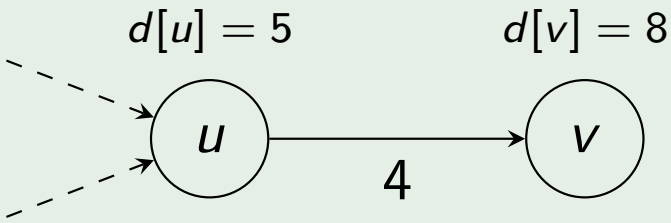
Las filas, de 0 a 5 aristas

El árbol que deja p



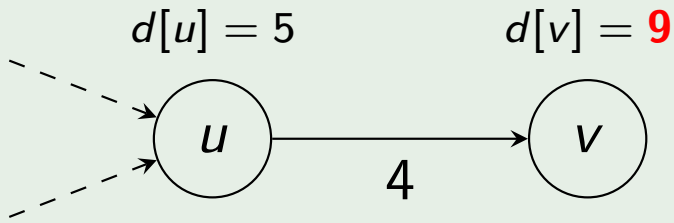
La relajación

No mejora



$5 + 4 = 9 \geq 8$: $d[v]$ se queda en 8.

Mejora



$5 + 4 = 9 < 11$: $d[v]$ baja de 11 a 9 y $p[v] = u$.

En el código de las cinco versiones

```
if(d[u] != INT_MAX && d[u] + peso < d[v]){
  d[v] = d[u] + peso;
  p[v] = u;
}
```

La condición `d[u] != INT_MAX` evita relajar desde un vértice al que todavía no se ha llegado.

$n - 1$ rondas y una más

Las rondas

En la ronda i se relajan todas las aristas. Después de $n - 1$ rondas, si no hay ciclos negativos alcanzables, $d[v] = \delta(s, v)$ para todo v.

La ronda adicional

Se recorren otra vez todas las aristas, sin relajar: si alguna (u, v) cumple $d[u] + w(u, v) < d[v]$, hay un ciclo negativo alcanzable desde s (CLRS, Teorema 24.4).

Pregunta

¿Por qué una arista que todavía se puede relajar después de $n - 1$ rondas delata un ciclo negativo?

$n - 1$ rondas y una más

Respuesta

Si no hubiera ciclo negativo alcanzable, d ya sería δ, y δ cumple la desigualdad triangular: $\delta(s, v) \leq \delta(s, u) + w(u, v)$ (CLRS, Lema 24.10). Ninguna arista se podría relajar.

En el grafo del ejemplo

La fila 5 es igual a la 4: ninguna arista se puede relajar y no hay ciclo negativo. El ciclo $\langle 3, 1, 2, 3 \rangle$ pesa $1 + 4 - 5 = 0$, que no es negativo.

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

Lo que comparten las cinco versiones

```
int n;
vector<vector<pair<int, int> > > G(500);
vector<int> p(500);
vector<vector<int>> d(500, vector<int>(500));
```

Las globales

- `G[u]`: la lista de adyacencia de u, con pares `(v, peso)`. Desde la versión 3 se llama `adj`.
- `d` y `p`: distancias y predecesores. Aquí `d` es una matriz de 500×500.
- La fuente es siempre 0: `main` llama a `bellmanFord(0)`.

La entrada

Una línea con n y m, y m líneas con `u v peso`, una por arista. Cada archivo trae su ejemplo comentado al final.

Lo que comparten las cinco versiones

```
int main(){
  int m, i, u, v, peso;

  cin >> n >> m;

  for(i = 0; i < m; i++){
    cin >> u >> v >> peso;
    G[u].push_back(make_pair(v, peso));
  }

  if(bellmanFord(0)){
    for(i = 0; i < n; ++i)
      cout << "Distancia a nodo " << i << ": " << d[n - 1][i] << endl;

    for(i = 0; i < n; ++i)
      cout << "Predecesor nodo " << i << ": " << p[i] << endl;
  }
  else
    cout << "El grafo tiene ciclos negativos" << endl;

  return 0;
}
```

bfm1: la matriz

```
bool bellmanFord(int s){
  int i, j, u, v, peso;
  bool ans = true;

  for(i = 0; i < n; ++i){
    p[i] = -1;
    for(j = 0; j < n; ++j)
      d[i][j] = INT_MAX;
  }

  d[0][s] = 0;
  for(i = 1; i <= n - 1; ++i){
    for(j = 0; j < n; ++j)
        d[i][j] = d[i - 1][j];
    for(u = 0; u < n; u++){
      for(j = 0; j < G[u].size(); ++j){
        v = G[u][j].first;
        peso = G[u][j].second;
        if(d[i - 1][u] != INT_MAX && d[i - 1][u] +
        ↪  peso < d[i - 1][v]){
          d[i][v] = d[i - 1][u] + peso;
          p[v] = u;
        }
      }
    }
  }
  u = 0;
  while(ans && u < n){
    i = 0;
    while(ans && i < G[u].size()){
      v = G[u][i].first;
      peso = G[u][i].second;
      if(d[n - 1][u] != INT_MAX && d[n - 1][u] +
      ↪  peso < d[n - 1][v])
        ans = false;
      i += 1;
    }
    u += 1;
  }

  return ans;
}
```

bfm1: qué hace cada parte

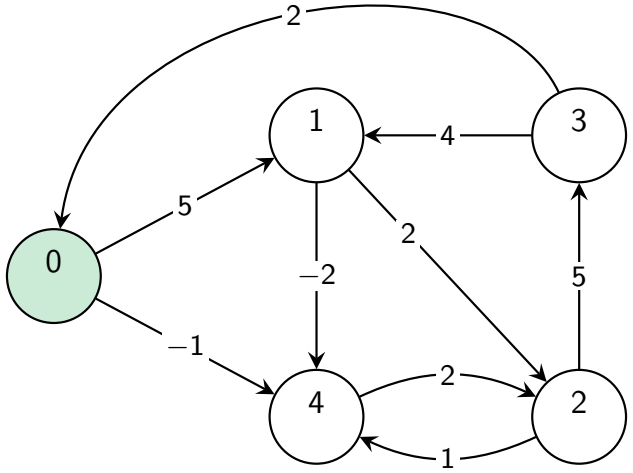
La fila i es d_i

- Todo arranca en `INT_MAX` y `d[0][s] = 0`: la fila 0.
- La ronda i copia la fila $i-1$ en la fila i y relaja cada arista *leyendo* la fila $i-1$ y *escribiendo* en la fila i. Un valor que cambió en esta ronda no se usa en la misma ronda.
- Después de la ronda $n-1$, la respuesta está en la fila $n-1$.

La revisión

Los dos `while` recorren todas las aristas sobre la fila $n-1$. La primera que se pueda relajar pone `ans` en `false` y detiene los dos ciclos.

bfm1 sobre el grafo del código



Las filas que deja la matriz, salidas de correr `bfm1` con el ejemplo comentado al final del archivo.

Fila	0	1	2	3	4
0	0	∞	∞	∞	∞
1	0	5	∞	∞	-1
2	0	5	1	∞	-1
3	0	5	1	6	-1
4	0	5	1	6	-1

bfm1 sobre el grafo del código

Lo que sale

$d = (0, 5, 1, 6, -1)$ y $p = (-1, 0, 4, 2, 0)$. Cada fila agrega exactamente una arista: 2 aparece en la fila 2 por $\langle 0, 4, 2 \rangle$ y 3 en la fila 3 por $\langle 0, 4, 2, 3 \rangle$.

Lo que cuesta

- Tiempo: llenar la matriz cuesta $\Theta(n^2)$; cada una de las $n - 1$ rondas copia una fila, $\Theta(n)$, y recorre las listas, $\Theta(n + m)$; la revisión, $O(n + m)$. Total: $\Theta(n^2 + nm)$.
- Espacio: $\Theta(n^2)$ por la matriz. Con 500 filas de 500 columnas se reservan 250 000 enteros, sea cual sea n.

Pregunta

Para llenar la fila i solo se lee la fila $i - 1$. ¿Para qué se guardan las demás?

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

bfm2: un solo vector

```
bool bellmanFord(int s){
  int i, j, u, v, peso;
  bool ans = true;

  for(i = 0; i < n; ++i){
    p[i] = -1;
    d[i] = INT_MAX;
  }

  d[s] = 0;
  for(i = 1; i <= n - 1; ++i){
    for(u = 0; u < n; u++){
      for(j = 0; j < G[u].size(); ++j){
        v = G[u][j].first;
        peso = G[u][j].second;
        if(d[u] != INT_MAX && d[u] + peso < d[v]){
          d[v] = d[u] + peso;
          p[v] = u;
        }
      }
    }
  }
  u = 0;
  while(ans && u < n){
    i = 0;
    while(ans && i < G[u].size()){
      v = G[u][i].first;
      peso = G[u][i].second;
      if(d[u] != INT_MAX && d[u] + peso < d[v])
        ans = false;
      i += 1;
    }
    u += 1;
  }

  return ans;
}
```

bfm2: lo que cambia

Respecto de bfm1

- `d` pasa a ser `vector<int> d(500)`.
- Desaparece la copia de la fila anterior.
- La relajación lee y escribe en el mismo vector: `d[u] + peso < d[v]`.

Pregunta

Ahora un valor que bajó en esta ronda sí se usa en la misma ronda. En la ronda 1, ¿se siguen calculando solo caminos de una arista?

bfm2: la ronda 1, arista por arista

Arista	0	1	2	3	4	¿Cambia?
inicio	0	∞	∞	∞	∞	
$(0,1)$, 5	0	5	∞	∞	∞	sí, $p[1] = 0$
$(0,4)$, -1	0	5	∞	∞	-1	sí, $p[4] = 0$
$(1,4)$, -2	0	5	∞	∞	-1	no: $5 - 2 = 3$
$(1,2)$, 2	0	5	7	∞	-1	sí, $p[2] = 1$
$(2,4)$, 1	0	5	7	∞	-1	no: $7 + 1 = 8$
$(2,3)$, 5	0	5	7	12	-1	sí, $p[3] = 2$
$(3,0)$, 2	0	5	7	12	-1	no
$(3,1)$, 4	0	5	7	12	-1	no
$(4,2)$, 2	0	5	1	12	-1	sí, $p[2] = 4$

Respuesta

No. Al terminar la ronda 1, $d[3] = 12$ viene de $\langle 0, 1, 2, 3\rangle$, de tres aristas, y $d[2] = 1$ de $\langle 0, 4, 2\rangle$, de dos. En la fila 1 de la matriz, 3 y 2 seguían en ∞.

bfm2: por qué sigue siendo correcta

Lo que sí vale después de la ronda i

$$\delta(s, v) \leq d[v] \leq d_i(v).$$

El vector llega a lo que tendría la fila i de la matriz, o a algo mejor, y nunca baja de δ, porque cada valor es el peso de un camino real. Después de $n - 1$ rondas, $d[v] = \delta(s, v)$.

Ronda	0	1	2	3	4
1	0	5	1	12	-1
2	0	5	1	6	-1
3	0	5	1	6	-1
4	0	5	1	6	-1

bfm2: por qué sigue siendo correcta

Lo que cuesta

- Tiempo: $n-1$ rondas de $\Theta(n+m)$ y la revisión: $\Theta(n(n+m))$, que es $\Theta(nm)$ cuando $m \geq n$.
- Espacio: $\Theta(n)$ para `d` y `p`, además del grafo.

Lo que se pierde

El vector ya no dice "a lo sumo i aristas". Cuando el problema limita el número de aristas, como UVa 11280 más adelante, se necesitan las filas.

Pregunta

Las rondas 3 y 4 no cambiaron nada. Antes de eso: si la revisión encuentra un ciclo negativo, ¿cómo se sabe cuál es?

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

bfm3: el ciclo

```
bool bellmanFord(int s){
  int i, j, u, v, peso, t, cur;
  bool ans = true, flag = true;

  for(i = 0; i < n; ++i){
    p[i] = -1;
    d[i] = INT_MAX;
  }

  d[s] = 0;
  i = 0;
  while(i < n){
    t = -1;
    for(v = 0; v < n; v++){
      for(j = 0; j < adj[v].size(); ++j){
        u = adj[v][j].first;
        peso = adj[v][j].second;
        if(d[v] != INT_MAX && d[u] > d[v] + peso){
          d[u] = d[v] + peso;
          p[u] = v;
          t = v;
        }
      }
    }
    ++i;
  }
  if(t != -1){
    for(i = 0; i < n; ++i)
      t = p[t];
    cur = t;
    ciclo = vector<int>();

    while(cur != t || ciclo.size() == 0){
      ciclo.push_back(cur);
      cur = p[cur];
      printf("aqui\n");
    }
    ans = false;
  }

  return ans;
}
```

bfm3: lo que cambia

Respecto de bfm2

- El ciclo hace n rondas: la ronda n hace de revisión.
- En la ronda, `t` guarda el origen de la última arista que cambió algo, o -1 si nada cambió. Aquí `v` es el origen de la arista y `u` el destino.
- Si después de la ronda n sigue `t != -1`, algo cambió en esa ronda: hay un ciclo negativo.
- Se sube n veces por `p` desde `t`, y desde ahí se recorre `p` hasta volver, anotando en `ciclo`.

Una línea de rastreo

`printf(.ªqui\n")` imprime una vez por cada vértice del ciclo; no cambia el resultado.

bfm3: lo que cambia

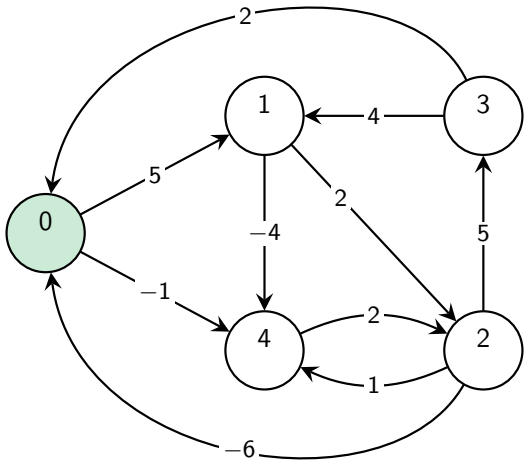
Por qué subir n veces

`t` puede colgar del ciclo sin estar en él. Al subir n veces se visitan $n+1$ vértices: alguno se repite, y desde la primera repetición la subida da vueltas al ciclo. Después de n pasos ya se está dentro.

Lo que cuesta

Tiempo: siempre n rondas, $\Theta(n(n+m))$, aunque nada cambie desde la segunda; recuperar el ciclo, $O(n)$. Espacio: $\Theta(n)$.

bfm3 sobre el segundo ejemplo



El ejemplo comentado al final de `bfm3.cpp`: $(1, 4)$ pesa -4 y aparece $(2, 0)$ con peso -6.

Ronda	0	1	2	3	4	t
1	0	5	1	12	−1	4
2	−5	5	1	6	−1	2
3	−5	0	−4	6	−6	4
4	−10	0	−4	1	−6	2
5	−10	−5	−9	1	−11	4

bfm3 sobre el segundo ejemplo

La recuperación

Al final, $p = (2, 0, 4, 2, 0)$ y $\texttt{t} = 4$. Subir cinco veces desde 4 pasa por $0, 2, 4, 0, 2$: `t` queda en 2. Desde ahí, `ciclo` recibe 2, 4 y 0 y para al volver a 2.

El ciclo

`ciclo` está al revés, porque se sube por p: el ciclo es $\langle 0, 4, 2, 0 \rangle$, de peso $-1 + 2 - 6 = -5$. El programa imprime 2, 4 y 0.

Pregunta

Las rondas 3 y 4 de bfm2 no cambiaron nada en el primer ejemplo. ¿Puede cambiar algo en una ronda que viene después de una ronda sin cambios?

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

Una ronda sin cambios

Respuesta

No. Una ronda solo lee `d` y la lista de aristas. Si una ronda no cambió nada, la siguiente lee los mismos valores, compara lo mismo y tampoco cambia nada; y así todas las que siguen.

Consecuencia

- Apenas una ronda termina sin cambios se puede parar: `d` ya tiene las distancias.
- La revisión sobra en ese caso: si nada cambió, ninguna arista se puede relajar.

bfm4: parar a tiempo

```
bool bellmanFord(int s){
  int i, j, u, v, peso, t;
  bool ans = true;

  for(i = 0; i < n; ++i){
    p[i] = -1;
    d[i] = INT_MAX;
  }

  d[s] = 0;
  t = 0;
  for(i = 1; i < n && t != -1; ++i){
    t = -1;
    for(v = 0; v < n; v++){
      for(j = 0; j < adj[v].size(); ++j){
        u = adj[v][j].first;
        peso = adj[v][j].second;
        if(d[v] != INT_MAX && d[v] + peso < d[u]){
          d[u] = d[v] + peso;
          p[u] = v;
          t = v;
        }
      }
    }
  }
  printf("Iteraciones realizadas: %d\n", i - 1);

  if(t != -1){
    u = 0;
    while(u < n){
      i = 0;
      while(ans && i < adj[u].size()){
        v = adj[u][i].first;
        peso = adj[u][i].second;
        if(d[u] + peso < d[v])
          ans = false;
        i += 1;
      }
      u += 1;
    }
  }

  return ans;
}
```

bfm4: lo que cambia y lo que ahorra

Respecto de bfm3

- Vuelven las $n-1$ rondas, ahora con la condición `i < n && t != -1`: el ciclo sigue mientras la ronda anterior haya cambiado algo.
- La revisión se hace solo si la última ronda cambió algo.
- Imprime cuántas rondas hizo.

Ronda	0	1	2	3	4	t
1	0	5	1	12	−1	4
2	0	5	1	6	−1	2
3	0	5	1	6	−1	−1

En el primer ejemplo imprime `Iteraciones realizadas: 3` y no hace la revisión.

bfm4: lo que cambia y lo que ahorra

Lo que se ahorró

bfm2 examinó $4 \times 9 = 36$ aristas en las rondas y 9 en la revisión: 45.
bfm4 examinó $3 \times 9 = 27$.

Lo que cuesta

- Tiempo: en el peor caso hace las $n - 1$ rondas, $O(n(n + m))$. En el mejor, una o dos rondas, $\Theta(n + m)$.
- Espacio: $\Theta(n)$.

Pregunta

En la ronda 2 solo cambió $d[3]$. ¿Hacía falta examinar las nueve aristas para descubrirlo? ¿Cuáles podían cambiar algo?

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

Solo las aristas que pueden cambiar algo

Respuesta

Relajar (u, v) solo puede cambiar algo si $d[u]$ bajó desde la última vez que se relajaron las aristas de u. En la ronda 2 solo había que mirar las aristas que salen de los vértices que cambiaron en la ronda 1.

La idea de Moore

Una cola con los vértices cuyo d bajó y cuyas aristas falta relajar. Se saca uno, se relajan sus aristas y cada vértice que baja entra a la cola.

Solo las aristas que pueden cambiar algo

```
int n;
vector<vector<pair<int, int> > > adj(50000);
vector<int> p(50000);
vector<int> d(50000);
vector<bool> enCola(50000);
vector<int> conteo(50000);
```

Dos arreglos nuevos

- `enCola[v]`: `true` si v ya está esperando. Un vértice que baja y ya está en la cola no se agrega otra vez: cuando salga usará el d más reciente.
- `conteo[v]`: cuántas veces ha entrado v a la cola. Si pasa de n, hay un ciclo negativo.

Los vectores son de 50 000: con espacio $\Theta(n)$ caben grafos mucho más grandes que con la matriz.

bfm5: la cola

```
bool bellmanFordOpt(int s){
  int i, j, u, v, peso;
  queue<int> cola;
  bool ans = true;

  for(i = 0; i < n; ++i){
    p[i] = -1;
    d[i] = INT_MAX;
    enCola[i] = false;
    conteo[i] = 0;
  }

  d[s] = 0;
  cola.push(s);
  enCola[s] = true;

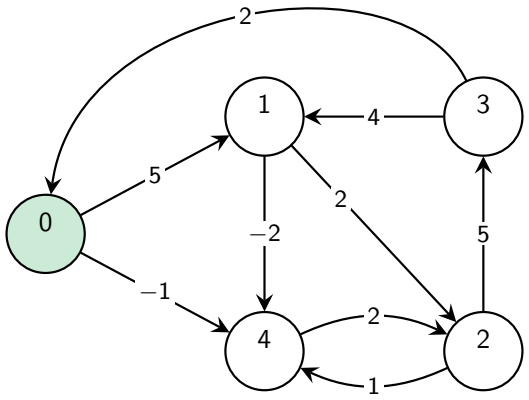
  while(ans && !cola.empty()){
    u = cola.front();
    cola.pop();
    enCola[u] = false;
    for(i = 0; i < adj[u].size(); ++i){
      v = adj[u][i].first;
      peso = adj[u][i].second;
      if(d[u] != INT_MAX && d[u] + peso < d[v]){
        d[v] = d[u] + peso;
        p[v] = u;

        if(!enCola[v]){
          cola.push(v);
          enCola[v] = true;
          conteo[v]++;

          //verifica si encontró un ciclo negativo
          if(conteo[v] > n)
            ans = false;
        }
      }
    }
  }

  return ans;
}
```

bfm5 sobre el grafo del código



Sale	Cambian	d					Cola	conteo				
		0	1	2	3	4		0	1	2	3	4
		0	∞	∞	∞	∞	0	0	0	0	0	0
0	1, 4	0	5	∞	∞	-1	1, 4	0	1	0	0	1
1	2	0	5	7	∞	-1	4, 2	0	1	1	0	1
4	2	0	5	1	∞	-1	2	0	1	1	0	1
2	3	0	5	1	6	-1	3	0	1	1	1	1
3		0	5	1	6	-1	vacía	0	1	1	1	1

bfm5 sobre el grafo del código

Pregunta

Cuando sale 4, $d[2]$ baja de 7 a 1, pero 2 no entra otra vez a la cola y `conteo[2]` no cambia. ¿Por qué?

bfm5 sobre el grafo del código

Respuesta

2 ya estaba esperando: entró cuando salió 1, y `enCola[2]` es `true`. Cuando salga usará $d[2] = 1$.

Lo que se examinó

Cinco salidas y $2 + 2 + 1 + 2 + 2 = 9$ aristas, contra 27 de bfm4 y 45 de bfm2. El resultado es el mismo: $d = (0, 5, 1, 6, -1)$ y $p = (-1, 0, 4, 2, 0)$.

Con el segundo ejemplo

Con el ciclo $\langle 0, 4, 2, 0 \rangle$ la cola no se vacía. En la salida 26 sale 0, `conteo[1]` llega a $6 > 5$ y la función devuelve `false`.

bfm5: por qué `conteo > n` y lo que cuesta

Por qué basta contar hasta n

Como la cola es FIFO, los vértices salen por tandas: la tanda 0 es $\{s\}$ y la tanda k son los que entraron mientras salía la tanda $k-1$. La tanda k consigue lo que conseguía la ronda k, o algo mejor, y por `enCola` un vértice entra a lo sumo una vez por tanda. Sin ciclos negativos alcanzables nada cambia después de la tanda $n-1$, así que ningún vértice entra más de n veces. Con un ciclo negativo la cola no se vacía y algún `conteo` pasa de n.

bfm5: por qué `conteo > n` y lo que cuesta

Lo que cuesta

- Tiempo: cada salida de u cuesta $O(1 + \text{grado}(u))$, y cada vértice sale a lo sumo unas n veces: $O(n(n+m))$ en el peor caso, como las rondas. La ganancia es que solo se examinan aristas que pueden cambiar algo.
- Espacio: $\Theta(n)$ para `d`, `p`, `enCola`, `conteo` y la cola.

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

Las cinco versiones juntas

Versión	Qué agrega	Tiempo	Espacio	Aristas en el ejemplo
bfm1	la matriz d_i	$\Theta(n^2 + nm)$	$\Theta(n^2)$	45
bfm2	un vector	$\Theta(n(n+m))$	$\Theta(n)$	45
bfm3	recupera el ciclo	$\Theta(n(n+m))$	$\Theta(n)$	45
bfm4	para sin cambios	$O(n(n+m))$	$\Theta(n)$	27
bfm5	cola de Moore	$O(n(n+m))$	$\Theta(n)$	9

La última columna cuenta las aristas examinadas con el ejemplo comentado de `bfm1.cpp`, de 5 vértices y 9 aristas; bfm3 hace 5 rondas y ninguna revisión aparte.

Cuál usar

- Si el problema limita el número de aristas: las filas de bfm1.
- Si hay que mostrar el ciclo: bfm3.
- En lo demás, bfm4 o bfm5.

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

UVa 321, The New Villa

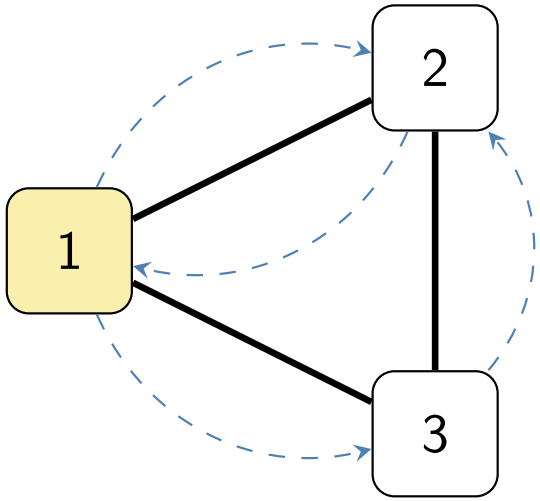
El problema

Una casa de $r \leq 10$ habitaciones: la 1 es el hall y la r el dormitorio. Hay puertas entre habitaciones e interruptores mal conectados: un interruptor de la habitación k prende o apaga la luz de la habitación l. Solo la luz del hall está prendida. El señor Black nunca entra a una habitación oscura ni apaga la luz de la habitación donde está. Debe llegar al dormitorio con solo esa luz prendida, en el menor número de pasos: moverse, prender y apagar cuentan uno cada uno.

Enunciado: `https://onlinejudge.org/external/3/321.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=257`

UVa 321, The New Villa



En negro, las puertas; en azul, cada interruptor va de donde está a la luz que controla.

La villa 1 del enunciado

$r = 3$. Puertas 1–2, 1–3, 3–2. Interruptores $1 \to 2$, $1 \to 3$, $2 \to 1$, $3 \to 2$.

Pregunta

Las puertas forman un grafo: ¿basta una búsqueda en amplitud desde 1 sobre las habitaciones?

Por qué el grafo de las puertas no basta

Respuesta

No. Sobre las puertas, el dormitorio queda a un paso del hall, pero al empezar la habitación 3 está oscura y no se puede entrar. La solución del enunciado tiene 6 pasos, y los tres primeros ocurren sin salir del hall: prender la luz 2, prender la luz 3, ir a 2, apagar la luz 1, ir a 3 y apagar la luz 2.

La misma habitación, tres situaciones

En el hall con solo la luz 1 no hay a dónde ir. Con las luces 1 y 2 ya se puede pasar a 2, y con las tres, a 2 o a 3. Lo que se puede hacer depende de la habitación *y* de las luces.

El grafo de estados

Definición

Un estado es un par (h, L): la habitación h donde está el señor Black y el conjunto L de luces prendidas. Hay una arista de un estado a otro si un paso permitido lleva del primero al segundo.

Las transiciones desde (h, L)

- Accionar un interruptor de h que controla la luz l, con $l \neq h$: lleva a $(h, L \cup \{l\})$ si $l \notin L$ y a $(h, L \setminus \{l\})$ si $l \in L$.
- Ir a una habitación vecina j con $j \in L$: lleva a (j, L).

El grafo de estados

Inicio y meta

Inicio: $(1, \{1\})$. Meta: $(r, \{r\})$.

Cuántos estados

r habitaciones por 2^r conjuntos de luces: con $r = 10$, a lo sumo $10 \cdot 1024 = 10\,240$. En la villa 1 hay $3 \cdot 2^3 = 24$, y la búsqueda alcanza 12.

Qué algoritmo

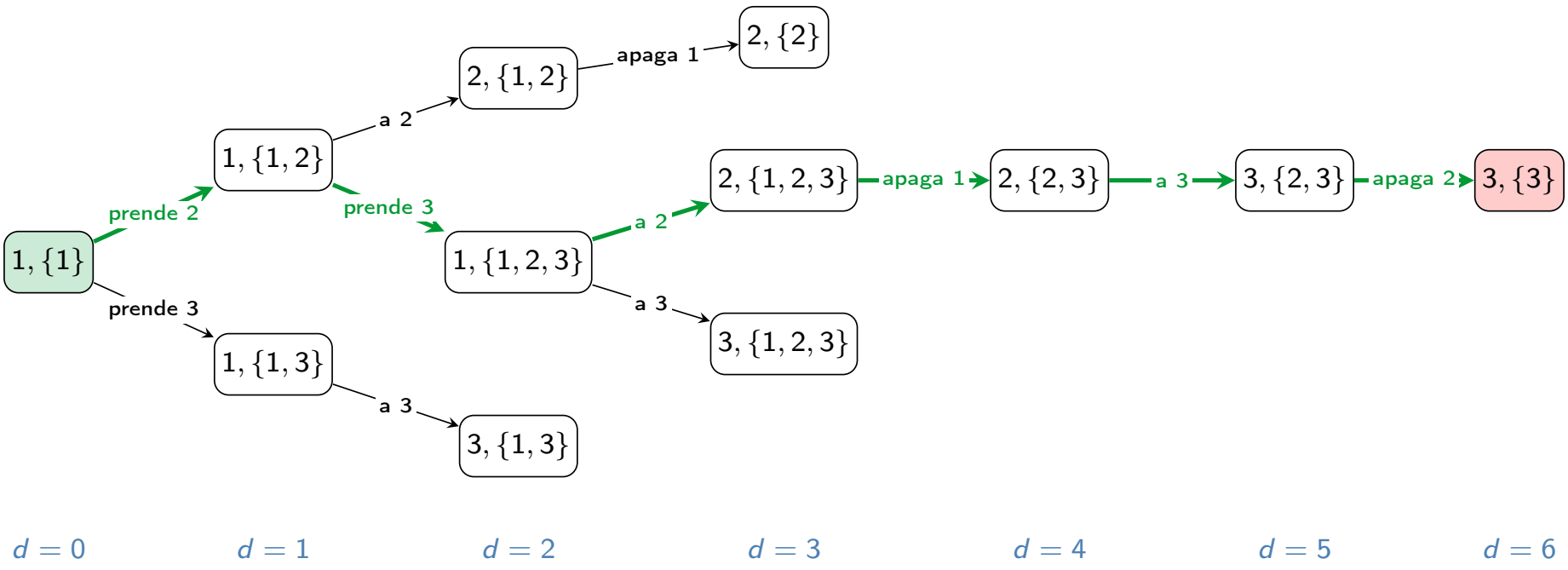
Todos los pasos cuestan 1: la búsqueda en amplitud desde el inicio da el camino con menos pasos.

El grafo de estados

El conjunto L en Python

Una tupla de booleanos: `luces[i]` dice si la luz i está prendida. Una tupla puede ser clave de un diccionario, y un estado es la tupla `(h, luces)`.

La villa 1 como grafo de estados



Los 12 estados que alcanza la búsqueda, por niveles, con las aristas del árbol que deja. En verde, el camino de 6 pasos hasta la meta.

Los sucesores de un estado

```
def sucesores(estado, puertas, interruptores):
  # Los estados a un paso de estado, cada uno con el texto de ese paso.
  (hab, luces) = estado
  res = []
  for l in interruptores[hab]:
    # nunca se apaga la luz de la habitacion donde se esta
    if l != hab:
      nuevas = list(luces)
      nuevas[l] = not nuevas[l]
      if nuevas[l]:
        texto = '- Switch on light in room ' + str(l) + '.'
      else:
        texto = '- Switch off light in room ' + str(l) + '.'
      res.append(((hab, tuple(nuevas)), texto))
  for j in puertas[hab]:
    # solo se entra a una habitacion con la luz prendida
    if luces[j]:
      res.append(((j, luces), '- Move to room ' + str(j) + '.'))
  return res
```

La lista nuevas es una copia de las luces: se cambia una posición y se vuelve a tupla.

La búsqueda en amplitud sobre los estados

```
def ssspBFS(s, puertas, interruptores):
  # d[u]: pasos de s a u; p[u]: (estado anterior, texto del paso).
  # Un estado que no esta en d no se ha descubierto: costo infinito.
  q = deque()
  d, p = {}, {}
  q.append(s)
  d[s] = 0

  while len(q) > 0:
    v = q.popleft()
    for (u, texto) in sucesores(v, puertas, interruptores):
      if u not in d:
        q.append(u)
        d[u] = d[v] + 1
        p[u] = (v, texto)

  return d, p
```

La búsqueda en amplitud sobre los estados

Lo mismo que sobre un grafo dado

Es `ssspBFS` con un solo origen. Los estados no son `0..n-1`, así que `d` y `p` son diccionarios, y `u not in d` hace las veces de `d[u] == float(ïnf")`. Los sucesores se calculan al sacar cada estado: el grafo nunca se construye completo.

Lo que cuesta

Cada estado entra una vez a la cola y genera a lo sumo $2r$ sucesores: $O(r \cdot 2^r \cdot r)$ pasos, sin contar la copia de las luces, que cuesta $O(r)$ por sucesor.

Del árbol a los pasos

```
def resolver(r, puertas, interruptores):
  # (hay, pasos): si se llega a la meta y el texto de cada paso.
  luces = [False] * (r + 1)
  luces[1] = True
  s = (1, tuple(luces))
  luces = [False] * (r + 1)
  luces[r] = True
  meta = (r, tuple(luces))
  d, p = ssspBFS(s, puertas, interruptores)
  hay = meta in d
  pasos = []
  if hay:
    x = meta
    while x != s:
      (anterior, texto) = p[x]
      pasos.append(texto)
      x = anterior
    pasos.reverse()
  return (hay, pasos)
```

Se sube por `p` desde la meta, como con un camino de la amplitud, y se voltea la lista. `main` lee las villas e imprime en el formato del enunciado.

Del árbol a los pasos

Con el ejemplo del enunciado

```
Villa #1
The problem can be solved in 6 steps:
- Switch on light in room 2.
- Switch on light in room 3.
- Move to room 2.
- Switch off light in room 1.
- Move to room 3.
- Switch off light in room 2.

Villa #2
The problem cannot be solved.
```

Igual, línea por línea, a la salida del enunciado.

Del árbol a los pasos

Consejo

El estado es la pieza que hay que encontrar: lo que hace falta saber para decidir qué pasos son posibles. La trampa está en $r = 1$: el hall ya es el dormitorio, y la respuesta es `The problem can be solved in 0 steps:`. El juez acepta cualquier solución con el menor número de pasos.

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

UVa 558, Wormholes

El problema

Hasta 1000 sistemas y 2000 agujeros de gusano dirigidos, cada uno con un salto en el tiempo t, con $-1000 \leq t \leq 1000$. Desde el sistema 0 se llega a todos. ¿Hay un ciclo que lleve indefinidamente al pasado?

Enunciado: `https://onlinejudge.org/external/5/558.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=499`

Cómo atacarlo

Un ciclo al pasado es un ciclo de peso negativo, y todos son alcanzables desde 0: basta `bellmanFord(0)` de bfm4 o bfm5 y se imprime `possible` si devuelve `false`. Dos trampas del código: los vectores globales de bfm4 tienen 500 posiciones y n llega a 1000, y como hay varios casos, cada lista de adyacencia se vacía antes de leer el siguiente.

UVa 11280, Flying to Fredericton

El problema

Hasta 100 ciudades y 1000 vuelos con costo; puede haber varios entre las mismas dos ciudades. Para cada consulta, un máximo de escalas: el costo más bajo de Calgary a Fredericton con a lo sumo esas escalas.

Enunciado: `https://onlinejudge.org/external/112/11280.pdf`

Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=2255`

Cómo atacarlo

k escalas son a lo sumo $k+1$ vuelos: la respuesta es d_{k+1} de Fredericton, una fila de la matriz de bfm1. Una matriz responde todas las consultas del caso; si $k+1$ pasa de $n-1$, se usa la fila $n-1$. El vector de bfm2 no sirve: los vuelos van de la ciudad más cercana a la más lejana, y en una sola ronda encadena varios.

UVa 11280, Flying to Fredericton

La trampa

Con dos vuelos a la misma ciudad, la fila i tiene que quedarse con el menor. Antes de escribir `d[i][v]` se compara contra el valor que ya tiene `d[i][v]`.

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

Errores comunes

Relajar desde `INT_MAX`

Sin `d[u] != INT_MAX`, la suma `INT_MAX + peso` se desborda: en C++ eso no está definido, y en la práctica da un número negativo que se toma como distancia.

Creer que la ronda i del vector es d_i

En la ronda 1 del ejemplo, bfm2 ya tenía un camino de tres aristas. Para un límite de aristas se necesitan las filas.

Leer `d` cuando hay ciclo

Si la función devuelve `false`, los números de `d` no son distancias.

Errores comunes

Recoger el ciclo desde `t` sin subir

Si `t` cuelga del ciclo, subir por `p` nunca vuelve a `t` y el `while` no termina.

Moore sin `enCola` o sin `conteo`

Sin `enCola` la cola se llena de copias. Sin `conteo`, un ciclo negativo deja el `while` corriendo para siempre.

Un ciclo negativo lejano

Un ciclo al que no se llega desde la fuente no cambia ninguna distancia, y las cinco versiones devuelven `true`.

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

Ejercicios

1. La matriz a mano

Sea A el grafo dirigido con $V = \{0, \ldots, 4\}$, fuente 0 y estas aristas, en este orden de entrada, como `u v peso`: `0 3 2`, `3 1 -1`, `1 4 3`, `0 2 6`, `2 1 -2`, `4 2 1`, `3 4 7`. Llene las filas 0 a 4 de la matriz de bfm1. ¿En qué fila aparece por primera vez el valor final de $d[2]$, y cuántas aristas tiene ese camino?

2. El vector sobre el mismo grafo

Ejecute bfm2 sobre A y escriba d al final de cada ronda. ¿En qué ronda llega $d[2]$ a su valor final? Explique por qué llega antes que en la matriz.

Ejercicios

3. Rondas contra cola

Sobre A: ¿cuántas rondas hace bfm4 y cuántas aristas examina? Haga la tabla de bfm5 con lo que sale, lo que cambia, d, la cola y `conteo`, y cuente las aristas que examina.

4. El ciclo y `t`

Sea C el grafo con $V = \{0, \ldots, 5\}$, fuente 0 y aristas `0 1 1`, `1 2 1`, `2 3 -4`, `3 1 1`, `3 4 2`, `4 5 2`. Ejecute bfm3: diga `t` después de la ronda 6, los vértices por los que pasa al subir 6 veces y lo que imprime. ¿Qué pasaría si `ciclo` se recogiera desde `t` sin subir?

Ejercicios

5. Una ronda sin cambios

Demuestre que si en la ronda i de bfm4 ningún d cambia, tampoco cambia ninguno en la ronda $i+1$, y que entonces ninguna arista se puede relajar. Escriba la demostración en cuatro partes: teorema, estrategia, desarrollo y conclusión.

6. Dos filas bastan

Modifique bfm1 para que guarde solo dos vectores, la fila anterior y la actual, con espacio $\Theta(n)$, y siga calculando d_k exacto. Dé un grafo de tres vértices en el que, después de la ronda 1, bfm2 tiene en algún $d[v]$ un valor distinto de $d_1(v)$.

Ejercicios

7. Otra villa

$r = 4$, puertas 1–2, 2–3, 2–4, e interruptores $1 \to 2$, $2 \to 1$, $2 \to 3$, $3 \to 1$, $3 \to 4$, $4 \to 2$.

- ¿Cuántos estados tiene el grafo? Liste los sucesores del estado inicial.
- Encuentre a mano una solución con el menor número de pasos.
- Corra la solución de la clase y diga cuántos estados alcanza la búsqueda.

Pistas

Ejercicios 1 a 4

1. La fila 4 es $(0, 1, 5, 2, 4)$. El camino a 2 es $\langle 0, 3, 1, 4, 2 \rangle$.
2. La ronda 1 deja $(0, 1, 6, 2, 9)$. Mire el orden en que se recorren las listas.
3. bfm4 hace 3 rondas. bfm5 saca 6 vértices y examina 8 aristas.
4. `t` no está en el ciclo. El ciclo pesa -2.

Pistas

Ejercicios 5 a 7

5. Se procede de forma directa: la ronda $i+1$ hace las mismas comparaciones que la ronda i, con los mismos valores.
6. Con dos aristas $(0,1)$ y $(1,2)$ basta, si la lista de 0 se recorre antes que la de 1.
7. Hay 64 estados y el inicial tiene un solo sucesor. La solución tiene 10 pasos: para apagar la luz 3 hay que volver a la habitación 2.

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

Lo que sigue

Grafos dirigidos sin ciclos

Bellman-Ford hace rondas porque no sabe en qué orden relajar las aristas. En un grafo dirigido acíclico ese orden existe, y ya se sabe calcular.

Para investigar

- CLRS, Sección 24.2: si las aristas se relajan una sola vez, vértice por vértice en orden topológico, ¿por qué basta una pasada, y cuánto cuesta?
- CLRS, Sección 24.3, el algoritmo de Dijkstra: si ningún peso es negativo, ¿qué vértice conviene sacar primero, y qué reemplaza a la cola de Moore?

Contenido

1 Antes de empezar

2 Caminos de a lo sumo k aristas

3 Versión 1: una matriz de n filas

4 Versión 2: un solo vector

5 Versión 3: recuperar el ciclo negativo

6 Versión 4: parar cuando una ronda no cambia nada

7 Versión 5: la cola de Moore

8 Las cinco versiones juntas

9 Grafos de estado

10 Problemas de juez

11 Errores comunes

12 Ejercicios

13 Lo que sigue

14 Referencias

Referencias


Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009. Sección 24.1, Lema 24.2 y Teorema 24.4 (Bellman-Ford); Sección 24.5, Lema 24.10 (desigualdad triangular).


Bellman, R. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.


Ford, L. R. *Network Flow Theory*. RAND Corporation, Paper P-923, 1956.


Moore, E. F. The shortest path through a maze. En *Proceedings of the International Symposium on the Theory of Switching*, Harvard University Press, 1959, pp. 285–292.


Halim, S., Halim, F., Effendy, S. *Competitive Programming 4*. Lulu, 2020. Capítulo 4: Bellman-Ford y su variante con cola; grafos de estados.


UVa Online Judge, problemas 321, 558 y 11280. `https://onlinejudge.org`